28

锁

在介绍并发的那一章里，我们看到了并发编程的一个基本问题：我们希望原子地执行一系列指令，但由于单处理器上存在中断（或者多个线程在多个处理器上并发执行），我们做不到。本章就来正面解决这个问题，为此要引入一种叫作**锁**（lock）的东西。程序员在源代码中用锁把临界区（critical section）包起来，从而确保每个这样的临界区都像一条原子指令一样执行。

28.1 锁的基本思想

举个例子，假设我们的临界区是下面这样，也就是对共享变量最典型的一种更新：

```
balance = balance + 1;
```

当然，临界区也可以是别的，比如向链表中添加一个元素，或者对共享结构做其他更复杂的更新；不过现在我们只看这个简单的例子。要使用锁，我们在临界区周围加上这样的代码：

```
1 lock_t mutex; // 某个全局分配的锁 'mutex'
2 ...
3 lock(&mutex);
4 balance = balance + 1;
5 unlock(&mutex);
```

锁就是一个变量，因此要使用锁，就必须声明某种**锁变量**（lock variable），比如上面的`mutex`。这个锁变量（简称“锁”）保存着锁在任一时刻的状态：要么是**可用的**（available，也说**未锁定**（unlocked）或**空闲**（free）），此时没有线程持有锁；要么是**已获取的**（acquired，也说**已锁定**（locked）或**被持有**（held）），此时恰好有一个线程持有锁，而且这个线程多半正处在临界区中。我们也可以在这个数据类型中保存其他信息，比如哪个线程持有锁，或者一个用来给获取锁的请求排序的队列，但这类信息对锁的使用者是隐藏的。

`lock()`和`unlock()`例程的语义很简单。调用`lock()`会尝试获取锁：如果没有其他线程持有锁（即锁是空闲的），这个线程就会获取锁，进入临界区；人们有时把这个线程叫作锁的**持有者**（owner）。如果此时另一个线程对同一个锁变量（本例中是`mutex`）调用`lock()`，那么只要锁还被别的线程持有，这次调用就不会返回；这样，在持有锁的第一个线程还待在临界区里时，其他线程就进不去。

锁的持有者一调用 unlock()，锁就又变成可用（空闲）的了。如果没有其他线程在等这个锁（即没有其他线程调用了 lock() 并卡在里面），锁的状态就只是变成空闲。如果有线程在等（卡在 lock() 里），其中一个线程（迟早）会注意到（或被告知）锁的状态变了，于是获取锁，进入临界区。

锁让程序员对调度有了最起码的一点控制权。一般来说，线程由程序员创建，却由操作系统按它自己选择的方式来调度。锁把一部分控制权交还给了程序员：在一段代码周围加上锁，程序员就能保证这段代码里任何时候最多只有一个线程在运行。因此，锁有助于把传统操作系统调度的一片混乱，变成一种更可控的活动。

28.2 Pthread 锁

POSIX 库把锁叫作**互斥锁**（mutex），因为它用来在线程之间实现**互斥**（mutual exclusion）：如果一个线程在临界区中，它就把其他线程挡在外面，直到它执行完这段临界区为止。因此，当你看到下面这段 POSIX 线程代码时，应该明白它做的事情和上面一样（我们同样用了自己的包装函数，在加锁和解锁时检查错误）：

```
1 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2
3 Pthread_mutex_lock(&lock); // 包装函数；失败时退出
4 balance = balance + 1;
5 Pthread_mutex_unlock(&lock);
```

你可能还注意到，POSIX 版本给加锁和解锁传入了一个变量，因为我们可能会用不同的锁来保护不同的变量。这样做可以提高并发度：与其用一把大锁，访问任何临界区时都用它（这是一种**粗粒度**（coarse-grained）的加锁策略），通常不如用不同的锁来保护不同的数据和数据结构，从而允许更多线程同时处在加了锁的代码中（这是一种**细粒度**（fine-grained）的方法）。

28.3 实现一个锁

到这里，你应该已经从程序员的角度对锁的工作方式有了一些了解。但锁应该怎样实现呢？需要什么硬件支持？需要什么操作系统支持？本章剩下的部分就来回答这一系列问题。

> 关键问题：如何实现锁
> 如何实现一个高效的锁？高效的锁能以较低的代价提供互斥，还可能具备下面要讨论的其他一些性质。需要什么硬件支持？需要什么操作系统支持？

要实现一个能用的锁，我们需要老朋友硬件的帮助，也需要好伙伴操作系统的帮助。多年来，各种计算机体系结构的指令集中加入了许多不同的硬件原语；我们不会研究这些指令是怎么实现的（毕竟，那是计算机体系结构课的内容），但会研究如何用它们来构建锁这样的互斥原语。我们还会研究操作系统如何参与进来，把整个图景补全，让我们能构建出一个完善的锁库。

 © 2008–25, ARPACI-DUSSEAU

28.4 评价锁

在动手实现锁之前，我们应该先弄清楚目标是什么，所以要问：怎样评价某种锁实现的效果？要评价一个锁能不能用（以及好不好用），应该先定下一些基本标准。第一条是锁能否完成它的基本任务，也就是提供**互斥**。说白了，就是这个锁管不管用，能不能阻止多个线程同时进入临界区？

第二条是**公平性**（fairness）。锁空闲时，每个争抢锁的线程是否都有公平的机会拿到它？换个角度，看看更极端的情况：有没有争抢锁的线程会**饿死**（starve），永远也拿不到锁？

最后一条标准是**性能**（performance），具体来说，是使用锁所增加的时间开销。这里有几种情况值得考虑。一种是没有争用（contention）的情况：只有一个线程在运行，它获取锁、再释放锁，开销是多少？另一种是多个线程在单个 CPU 上争抢锁的情况：这时有没有性能问题？最后，当涉及多个 CPU、每个 CPU 上都有线程在争抢锁时，锁的表现又如何？比较这几种不同的场景，我们就能更好地理解下面介绍的各种加锁技术对性能有什么影响。

28.5 控制中断

最早用来提供互斥的方案之一，是在临界区内关闭中断；这个方案是为单处理器系统发明的。代码大概是这样的：

```
1 void lock() {
2     DisableInterrupts();
3 }
4 void unlock() {
5     EnableInterrupts();
6 }
```

假设我们运行在这样一个单处理器系统上。在进入临界区之前（用某种特殊的硬件指令）关闭中断，就能确保临界区内的代码不会被中断，从而像原子操作一样执行。执行完之后，我们再（同样通过一条硬件指令）重新打开中断，程序照常往下执行。

这种方法的主要优点是简单。你肯定不用绞尽脑汁就能想明白它为什么管用。没有中断，线程就可以确信自己执行的代码会一直执行下去，不会有其他线程来干扰。

可惜，它的缺点很多。首先，这种方法要求我们允许任何调用它的线程执行特权操作（打开和关闭中断），因此必须信任这个功能不会被滥用。你已经知道，每当我们不得不信任任意一个程序时，多半就要有麻烦了。这里的麻烦体现在很多方面：一个贪心的程序可以在开始执行时就调用 `lock()`，从而独占处理器；更糟糕的是，一个出了错或者怀有恶意的程序可以调用 `lock()`，然后进入死循环。在后一种情况下，操作系统永远也拿不回系统的控制权，只剩一个办法：重启系统。把关中断当作通用的同步方案，需要对应用程序有太多的信任。

其次，这种方法在多处理器上不管用。如果多个线程在不同的 CPU 上运行，而且都想进入同一个临界区，那么关不关中断都无济于事；线程照样可以在其他处理器上运行，因而还是可能进入临界区。如今多处理器已经很普遍，我们的通用方案必须比这更好。

```
1  typedef struct __lock_t { int flag; } lock_t;
2
3  void init(lock_t *mutex) {
4      // 0 -> 锁可用，1 -> 锁被持有
5      mutex->flag = 0;
6  }
7
8  void lock(lock_t *mutex) {
9      while (mutex->flag == 1) // 测试（TEST）标志
10         ; // 自旋等待（什么也不做）
11     mutex->flag = 1;         // 现在设置（SET）它!
12 }
13
14 void unlock(lock_t *mutex) {
15     mutex->flag = 0;
16 }
```

图 28.1：**第一次尝试：一个简单的标志**

第三，长时间关闭中断可能导致中断丢失，进而引发严重的系统问题。例如，想象一下 CPU 没有注意到磁盘设备已经完成了一个读请求。操作系统又怎么知道该去唤醒等待这次读操作的进程呢?

出于这些原因，只有在少数场合才会把关中断当作互斥原语来用。例如，在某些情况下，操作系统本身会用屏蔽中断的办法，在访问自己的数据结构时保证原子性，或者至少防止出现某些棘手的中断处理情况。这种用法是合理的，因为在操作系统内部不存在信任问题，操作系统反正总是信任自己能执行特权操作。

28.6 失败的尝试：只用加载和存储

要想不依赖中断来实现一个像样的锁，我们就得依靠 CPU 硬件以及它提供的指令。我们先试着只用一个标志变量来实现一个简单的锁。在这次失败的尝试中，我们会看到实现锁所需的一些基本思路，并且（但愿能）明白，为什么只用一个变量、靠普通的加载和存储来访问它是不够的。

在这第一次尝试中（图 28.1），思路很简单：用一个简单的变量（`flag`）表示是否有线程占有锁。第一个进入临界区的线程会调用 `lock()`，它**测试**标志是否等于 1（这时不等于），然后把标志**设置**为 1，表示这个线程现在**持有**锁。线程执行完临界区后，调用 `unlock()` 清除标志，表示锁不再被持有。

如果在第一个线程处于临界区时，另一个线程恰好调用了 `lock()`，它就只会在 `while` 循环中**自旋等待**（spin-wait），等第一个线程调用 `unlock()` 清除标志。第一个线程一清除标志，等待的线程就会跳出 `while` 循环，替自己把标志设为 1，然后进入临界区。

可惜，这段代码有两个问题：一个是正确性问题，另一个是性能问题。只要你习惯了思考并发编程，正确性问题很容易看出来。想象一下图 28.2 中的代码交错执行，假设一开始 `flag=0`。

线程 1	**线程 2**
调用 `lock()`	
while (flag == 1)	
中断：切换到线程 2	
	调用 `lock()`
	while (flag == 1)
	flag = 1;
	中断：切换到线程 1
flag = 1; // 把标志设为 1（这边也设了!）	

图 28.2：**跟踪：没能实现互斥**

从这个交错执行可以看出，只要中断来得“及时”（还是说不合时宜?），我们很容易就能造出这样一种情况：两个线程都把标志设成了 1，于是两个线程都能进入临界区。这种行为就是专业人士所说的“糟糕”——我们显然没能满足最基本的要求：互斥。

性能问题我们后面再细说，它出在线程等待已被持有的锁的方式上：线程不停地检查 `flag` 的值，这种技术叫作**自旋等待**（spin-waiting）。自旋等待把时间都浪费在等另一个线程释放锁上了。在单处理器上，这种浪费尤其严重，因为等待者所等的那个线程根本无法运行（至少在发生上下文切换之前是这样）！因此，在接下来设计更精巧的方案时，我们也应该考虑如何避免这种浪费。

28.7 用测试并设置实现可用的自旋锁

关中断在多处理器上不管用，只用加载和存储的简单方法（如上所示）也行不通，于是系统设计者开始发明支持加锁的硬件机制。最早的多处理器系统，比如 20 世纪 60 年代初的 Burroughs B5000 [M82]，就有这样的支持；如今所有系统都提供这类支持，单 CPU 系统也不例外。

最容易理解的一种硬件支持，叫作**测试并设置**（test-and-set，TAS）指令，也叫**原子交换**（atomic exchange）[1]指令。下面这段 C 代码定义了测试并设置指令做的事情：

```
1 int TestAndSet(int *old_ptr, int new) {
2     int old = *old_ptr; // 取出 old_ptr 处的旧值
3     *old_ptr = new;     // 把 'new' 存入 old_ptr
4     return old;         // 返回旧值
5 }
```

测试并设置指令做的事情如下：它返回 `old_ptr` 所指向的旧值，同时把这个值更新为 `new`。当然，关键在于这一系列操作是**原子地**（atomically）执行的。之所以叫“测试并设置”，是因为它让你在“测试”旧值（也就是返回的那个值）的同时，把这个内存位置“设置”为新值；这条指令只比加载、存储稍强一点，却足以实现一个简单的**自旋锁**（spin lock），我们现在就来看图 28.3。或者更好的办法是：先自己想一想！

[1]每种支持测试并设置的体系结构对它的叫法都不一样。在 SPARC 上，它叫作 load/store unsigned byte 指令（`ldstub`）；在 x86 上，它是加锁版本的原子交换指令（`xchg`）。

补充：Dekker 算法和 Peterson 算法

20 世纪 60 年代，Dijkstra 向他的朋友们提出了并发问题，其中一位名叫 Theodorus Jozef Dekker 的数学家想出了一个解决方案 [D68]。我们这里讨论的方案要用特殊的硬件指令，甚至还要操作系统支持；**Dekker 算法**（Dekker's algorithm）则不同，它只用加载和存储（前提是它们彼此之间是原子的，这在早期硬件上成立）。

后来，Peterson 改进了 Dekker 的方法 [P81]。这里同样只用加载和存储，思路是确保两个线程永远不会同时进入临界区。下面是（针对两个线程的）**Peterson 算法**（Peterson's algorithm），看看你能不能看懂这段代码。`flag` 和 `turn` 这两个变量是做什么用的?

```
int flag[2];
int turn;

void init() {
    // 用 'flag' 表示你打算持有锁
    flag[0] = flag[1] = 0;
    // 轮到谁了？（线程 0 还是 1）
    turn = 0;
}
void lock() {
    // 'self' 是调用者的线程 ID
    flag[self] = 1;
    // 把机会让给另一个线程
    turn = 1 - self;
    while ((flag[1-self] == 1) && (turn == 1 - self))
        ; // 没轮到你时就自旋等待
}
void unlock() {
    // 撤销你的意图即可
    flag[self] = 0;
}
```

不知为何，有一阵子，开发不需要特殊硬件支持的锁风靡一时，给搞理论的人出了很多题目。当然，人们意识到假设有一点硬件支持要容易得多（何况这种支持在多处理器的早期就已经有了）之后，这方面的工作也就变得没什么用了。此外，上面这类算法在现代硬件上并不管用（因为现代硬件采用宽松内存一致性模型，relaxed memory consistency model），这让它们比以前更没用了。又一项研究被扔进了历史的垃圾堆……

我们来确认一下自己真的明白这个锁为什么管用。先设想一种情况：一个线程调用 `lock()`，而当前没有其他线程持有锁，因此 `flag` 应该是 0。这个线程调用 `TestAndSet(flag, 1)` 时，该例程会返回 `flag` 的旧值，也就是 0；于是，正在测试 `flag` 值的调用线程不会卡在 `while` 循环里自旋，而是直接拿到锁。这个线程还会原子地把这个值设置为 1，表示锁现在被持有了。线程执行完临界区后，调用 `unlock()`，把标志重新设为零。

 © 2008–25, ARPACI-DUSSEAU

```
1  typedef struct __lock_t {
2      int flag;
3  } lock_t;
4
5  void init(lock_t *lock) {
6      // 0：锁可用，1：锁被持有
7      lock->flag = 0;
8  }
9
10 void lock(lock_t *lock) {
11     while (TestAndSet(&lock->flag, 1) == 1)
12         ; // 自旋等待（什么也不做）
13 }
14
15 void unlock(lock_t *lock) {
16     lock->flag = 0;
17 }
```

图 28.3：**用测试并设置实现的简单自旋锁**

第二种情况是：已经有一个线程持有了锁（即 `flag` 为 1）。这时，第二个线程调用 `lock()`，同样会调用 `TestAndSet(flag, 1)`。这一次，`TestAndSet()` 会返回 `flag` 处的旧值 1（因为锁被持有），同时又一次把它设为 1。只要锁还被另一个线程持有，`TestAndSet()` 就会一次次返回 1，于是这个线程会一直自旋，直到锁最终被释放。等另一个线程终于把标志设为 0，这个线程再调用 `TestAndSet()` 时，就会得到 0，同时原子地把值设为 1，从而获取锁，进入临界区。

把（对旧锁值的）**测试**和（对新值的）**设置**合成一个原子操作，我们就确保了只有一个线程能获取锁。一个能用的互斥原语就是这样实现的！

现在你也许明白，为什么这类锁通常被称为**自旋锁**了。它是最容易实现的一种锁，只是一直自旋、消耗 CPU 周期，直到锁可用为止。要在单处理器上正确工作，它需要一个**抢占式调度程序**（preemptive scheduler），也就是会时不时通过定时器中断一个线程、转而运行另一个线程的调度程序。没有抢占，自旋锁在单 CPU 上就没多大意义，因为在 CPU 上自旋的线程永远不会让出 CPU。

提示：把并发想象成恶意的调度程序

从这个例子中，你也许能体会到理解并发执行所需的思路。你应该试着假装自己是一个**恶意的调度程序**（malicious scheduler），专挑最不合时宜的时刻中断线程，好让它们构建同步原语的拙劣尝试统统落空。你真是个刻薄的调度程序！某种特定的中断顺序也许不太可能出现，但它有可能出现，而要证明某种方法行不通，这就够了。用恶意的眼光思考问题是有用的！（至少有时候是这样。）

28.8 评价自旋锁

有了基本的自旋锁, 就可以按前面说过的几个维度来评价它的效果了。锁最重要的方面是**正确性**（correctness）：它能不能提供互斥？答案是能：自旋锁每次只允许一个线程进入临界区。因此，我们得到了一个正确的锁。

下一个维度是**公平性**。自旋锁对等待中的线程有多公平?你能保证等待的线程最终一定能进入临界区吗? 很遗憾，答案并不乐观：自旋锁不提供任何公平性保证。事实上，在有争用的情况下, 一个自旋的线程可能会永远自旋下去。简单的自旋锁（就是到目前为止讨论的这种）是不公平的，可能导致饥饿。

最后一个维度是**性能**。使用自旋锁的代价是什么? 为了更仔细地分析这一点, 我们建议考虑几种不同的情况。第一种，设想多个线程在单个处理器上争抢锁；第二种，设想线程分布在多个 CPU 上。

在单 CPU 的情况下，自旋锁的性能开销可能相当惨重。设想持有锁的线程在临界区内被抢占了。调度程序接下来可能会把其他线程挨个运行一遍（假设还有 $N-1$ 个），而每个线程都会试图获取锁。在这种情况下，这些线程在放弃 CPU 之前，都会把一整个时间片耗在自旋上，白白浪费 CPU 周期。

然而，在多个 CPU 上，自旋锁的效果还算不错（前提是线程数和 CPU 数大致相当）。思路是这样的：设想线程 A 在 CPU 1 上，线程 B 在 CPU 2 上，两者都在争抢一个锁。如果线程 A（CPU 1）拿到了锁，然后线程 B 也来尝试，B 就会（在 CPU 2 上）自旋。不过，临界区多半很短，因此锁很快就会变得可用，并被线程 B 获取。在这种情况下，自旋等待另一个处理器上的线程所持有的锁，不会浪费多少周期，因此可能是有效的。

28.9 比较并交换

有些系统还提供另一种硬件原语，叫作**比较并交换**（compare-and-swap，CAS）指令（比如在 SPARC 上就叫这个名字），在 x86 上则叫 **compare-and-exchange**。这条指令的 C 伪代码见图 28.4。

比较并交换的基本思路是：测试 `ptr` 所指地址上的值是否等于 `expected`；如果相等，就把 `ptr` 指向的内存位置更新为新值；如果不相等，什么也不做。不管哪种情况，都返回这个内存位置上原来的值，这样调用比较并交换的代码就能知道操作是否成功。

有了比较并交换指令, 我们可以用和测试并设置非常相似的方式实现锁。例如, 只要把上面的 `lock()` 例程换成下面这样就行：

```
1 int CompareAndSwap(int *ptr, int expected, int new) {
2     int original = *ptr;
3     if (original == expected)
4         *ptr = new;
5     return original;
6 }
```

图 28.4：**比较并交换**

```
1 void lock(lock_t *lock) {
2     while (CompareAndSwap(&lock->flag, 0, 1) == 1)
3         ; // 自旋
4 }
```

其余代码和上面测试并设置的例子相同。这段代码的工作方式也很相似：它只是检查标志是否为 0，如果是，就原子地换入 1，从而获取锁。在锁被持有期间试图获取锁的线程，会一直卡在那里自旋，直到锁最终被释放。

想看看怎样真正写出一个能从 C 中调用的 x86 版比较并交换，可以参考这段代码（出自 [S05]）[2]。

最后，你可能已经感觉到了，比较并交换是比测试并设置更强大的指令。以后我们简要探讨**无锁同步**（lock-free synchronization）[H91] 这类话题时，会用到它的这种能力。不过，如果只是用它实现一个简单的自旋锁，它的行为和我们上面分析过的自旋锁完全一样。

28.10 链接加载和条件存储

有些平台提供一对配合使用的指令，帮助构建临界区。例如，在 MIPS 体系结构 [H93] 上，可以把**链接加载**（load-linked）和**条件存储**（store-conditional）指令（合称 LL/SC）配合起来，构建锁和其他并发结构。这两条指令的 C 伪代码见图 28.5。Alpha、PowerPC 和 ARM 也提供类似的指令 [W09]。

链接加载和普通的加载指令很像，只是从内存中取出一个值，放进寄存器。关键区别在于条件存储：只有在此期间没有别的存储操作写过这个地址，它才会成功（并更新刚才链接加载的那个地址上的值）。如果成功，条件存储返回 1，并把 `ptr` 处的值更新为 `value`；如果失败，`ptr` 处的值不会被更新，并返回 0。

给自己一个挑战：想一想怎样用链接加载和条件存储来实现锁。想好之后，再看下面的代码，它给出了一种简单的解法。动手试试吧！答案在图 28.6 中。

`lock()` 的代码是唯一有意思的部分。首先，线程自旋等待标志变成 0（表示锁没有被持有）。标志一变成 0，线程就尝试通过条件存储获取锁；如果成功，线程就原子地把标志的值改成了 1，于是可以进入临界区。

注意条件存储是怎样失败的。一个线程调用 `lock()`，执行链接加载，由于锁没有被持有，得到的是 0。它还没来得及尝试条件存储就被中断了，另一个线程进入加锁代码，同样执行链接加载，也得到 0，并继续往下走。这时，两个线程都执行完了链接加载，都要尝试条件存储。这两条指令的关键特性在于，只有其中一个线程能成功把标志更新为 1，从而获取锁；第二个尝试条件存储的线程会失败（因为另一个线程在它的链接加载和条件存储之间更新了 `flag` 的值），只好再去尝试获取锁。

几年前在课堂上，本科生 David Capel 为喜欢布尔条件短路求值的读者，给上面的代码想出了一种更简洁的写法。看看你能不能想明白它为什么是等价的。它确实更短！

[2] github.com/remzi-arpacidusseau/ostep-code/tree/master/threads-locks

```
int LoadLinked(int *ptr) {
    return *ptr;
}

int StoreConditional(int *ptr, int value) {
    if (no update to *ptr since LL to this addr) {
        *ptr = value;
        return 1; // 成功!
    } else {
        return 0; // 更新失败
    }
}
```

图 28.5：**链接加载和条件存储**

```
void lock(lock_t *lock) {
    while (1) {
        while (LoadLinked(&lock->flag) == 1)
            ; // 自旋，直到它变成零
        if (StoreConditional(&lock->flag, 1) == 1)
            return; // 如果成功设为 1：完成
                    // 否则：再试一次
    }
}

void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

图 28.6：**用 LL/SC 实现锁**

```
void lock(lock_t *lock) {
    while (LoadLinked(&lock->flag) ||
           !StoreConditional(&lock->flag, 1))
        ; // 自旋
}
```

28.11 获取并增加

最后一种硬件原语是**获取并增加**（fetch-and-add，FAA）指令，它原子地把某个地址上的值加一，同时返回这个地址上的旧值。获取并增加指令的 C 伪代码是这样的：

```
int FetchAndAdd(int *ptr) {
    int old = *ptr;
    *ptr = old + 1;
    return old;
}
```

提示：代码越少越好（Lauer 定律）

程序员往往爱吹嘘自己为了做某件事写了多少代码。这从根本上就搞错了。真正值得吹嘘的，是完成一项任务只用了多么少的代码。简短、精炼的代码总是更受欢迎：它多半更容易理解，bug 也更少。正如 Hugh Lauer 在谈到构建 Pilot 操作系统时所说："如果让同一批人拥有两倍的时间，他们只用一半的代码就能做出同样好的系统。"[L81] 我们把这称为 **Lauer 定律**（Lauer's Law），它非常值得记住。所以，下次你吹嘘自己为了完成作业写了多少代码时，再想一想，或者更好的做法是：回去重写，让代码尽可能清晰、简洁。

在这个例子中，我们要用获取并增加实现一种更有意思的**票锁**（ticket lock，也叫排队自旋锁），它由 Mellor-Crummey 和 Scott 提出 [MS91]。加锁和解锁的代码见图 28.7（第 11 页）。

这个方案不再只用一个值，而是把 ticket 和 turn 两个变量结合起来构建锁。基本操作很简单：线程想获取锁时，先对 ticket 的值做一次原子的获取并增加；得到的值就当作这个线程的"轮次"（`myturn`）。然后用全局共享的 `lock->turn` 判断轮到了哪个线程；当某个线程的 (`myturn == turn`) 成立时，就轮到这个线程进入临界区了。解锁只需把 turn 加一，这样下一个等待的线程（如果有的话）就可以进入临界区了。

注意，这个方案和之前的尝试有一个重要的不同：它保证所有线程都能向前推进。一个线程一旦分到了自己的 ticket 值，在将来的某个时刻就一定会被调度（等排在它前面的线程都通过了临界区、释放了锁之后）。之前的尝试都没有这样的保证；例如，一个在测试并设置上自旋的线程，可能在其他线程不断获取、释放锁的同时，永远自旋下去。

```
1  typedef struct __lock_t {
2      int ticket;
3      int turn;
4  } lock_t;
5
6  void lock_init(lock_t *lock) {
7      lock->ticket = 0;
8      lock->turn   = 0;
9  }
10
11 void lock(lock_t *lock) {
12     int myturn = FetchAndAdd(&lock->ticket);
13     while (lock->turn != myturn)
14         ; // 自旋
15 }
16
17 void unlock(lock_t *lock) {
18     lock->turn = lock->turn + 1;
19 }
```

图 28.7：**票锁**

28.12 自旋太多：怎么办?

我们这些基于硬件的锁很简单（只有几行代码），而且管用（你愿意的话，甚至可以写点代码来证明这一点），对任何系统或代码来说，这都是两个极好的特性。然而，在某些情况下，这些方案可能相当低效。设想你在单个处理器上运行两个线程。再设想其中一个线程（线程 0）正处在临界区中，因而持有锁，而它不巧被中断了。第二个线程（线程 1）现在试图获取锁，却发现锁被持有。于是，它开始自旋。一直自旋。然后接着自旋。最后，时钟中断终于来了，线程 0 重新运行并释放了锁；最终（比如说等线程 1 下次运行时），线程 1 就不用再自旋那么久，能获取锁了。因此，每当一个线程在这种情况下卡住自旋，它就浪费了一整个时间片，什么也没干，只是在反复检查一个根本不会变的值！如果有 N 个线程争抢一个锁，问题会更严重：可能有 $N-1$ 个时间片以类似的方式被浪费掉，这些线程都只是在自旋，等着某一个线程释放锁。于是，我们的下一个问题是：

关键问题：如何避免自旋
怎样才能实现一个不会白白在 CPU 上自旋、浪费时间的锁?

单靠硬件支持解决不了这个问题。我们还需要操作系统的支持！现在就来看看具体可以怎样做。

28.13 简单的办法：让出来就行，宝贝

硬件支持已经帮我们解决了不少问题：锁能用了，甚至（像票锁那样）做到了获取锁时的公平。然而，我们还有一个问题：如果在临界区中发生了上下文切换，其他线程开始无休止地自旋，等着那个被中断的（持有锁的）线程重新运行，该怎么办?

我们的第一个尝试是一种简单而友好的方法：当你要自旋时，干脆把 CPU 让给另一个线程。正如 Al Davis 可能会说的：“让出来就行，宝贝！”[D91]。图 28.8（第 12 页）展示了这种方法。

```
1  void init() {
2      flag = 0;
3  }
4
5  void lock() {
6      while (TestAndSet(&flag, 1) == 1)
7          yield(); // 让出 CPU
8  }
9
10 void unlock() {
11     flag = 0;
12 }
```

图 28.8：**使用测试并设置和让出的锁**

在这种方法中，我们假设有一个操作系统原语 `yield()`，线程想放弃 CPU、让另一个线程运行时，就可以调用它。一个线程可以处于三种状态之一：**运行**（running）、**就绪**（ready）或阻塞（blocked）。`yield` 只是一个系统调用，它把调用者从运行状态移到就绪状态，从而让另一个线程进入运行状态。因此，让出的线程实际上是**取消了自己的调度**（deschedule）。

想想两个线程在一个 CPU 上的例子；在这种情况下，我们这种基于让出的方法效果相当好。如果一个线程碰巧调用了 `lock()`，发现锁被持有，它就会让出 CPU，于是另一个线程就会运行，并执行完它的临界区。在这个简单的情况下，让出的方法很管用。

现在来考虑有很多线程（比如 100 个）反复争抢一个锁的情况。这时，如果一个线程获取了锁，又在释放之前被抢占，其他 99 个线程都会调用 `lock()`，发现锁被持有，然后让出 CPU。假设采用某种时间片轮转（round-robin）调度程序，那么在持有锁的线程再次运行之前，这 99 个线程每个都要把这种“运行—让出”的模式走一遍。虽然这比自旋的方法要好（自旋会白白浪费 99 个时间片），但代价仍然很高：上下文切换的开销可能相当大，因此浪费仍然不少。

更糟糕的是，这种方法没有解决饥饿问题。一个线程可能陷入无休止的让出循环，其他线程却一次次进出临界区。显然，我们需要一种能直接解决饥饿问题的方法。

28.14 使用队列：用睡眠代替自旋

前面一些方法（票锁除外）真正的问题在于，它们把太多事情交给了运气。下一个运行哪个线程由调度程序决定；如果调度程序选得不好，运行的线程要么得自旋等待锁（我们的第一种方法），要么得立刻让出 CPU（我们的第二种方法）。不管哪种方式，都有可能造成浪费，而且都无法防止饥饿。

因此，当前持有者释放锁之后由哪个线程接着获取锁，我们必须明确地加以控制。为此，我们需要再多一点操作系统支持，还需要一个队列，记录哪些线程在等待获取锁。

为简单起见，我们使用 Solaris 提供的支持，它包括两个调用：`park()` 让调用它的线程进入睡眠，`unpark(threadID)` 唤醒由 `threadID` 指定的那个线程。把这两个例程配合起来，可以构建这样一种锁：调用者想获取一个已被持有的锁时，就让它睡眠；等锁空闲了再把它唤醒。我们来看图 28.9 中的代码，了解这类原语的一种可能用法。

我们在这个例子中做了几件有意思的事。第一，我们把测试并设置这个老办法和一个显式的锁等待者队列结合起来，构造了一个更高效的锁。第二，我们用这个队列来帮助控制下一个由谁获得锁，从而避免饥饿。

你可能注意到了 guard 的用法（图 28.9，第 14 页）：它基本上就是一个自旋锁，保护着这个锁内部对标志和队列的操作。因此，这种方法并没有完全避免自旋等待：一个线程可能在获取或释放锁时被中断，导致其他线程自旋等待它再次运行。不过，自旋所花的时间相当有限（只是加锁和解锁代码中的几条指令，而不是用户定义的临界区），因此这种方法也许是合理的。

你可能还注意到，在 `lock()` 中，当线程无法获取锁（锁已被持有）时，我们会小心地把自己加入队列（调用 `gettid()` 函数获取当前线程的线程 ID），把 guard 设为 0，然

```
1  typedef struct __lock_t {
2      int flag;
3      int guard;
4      queue_t *q;
5  } lock_t;
6
7  void lock_init(lock_t *m) {
8      m->flag  = 0;
9      m->guard = 0;
10     queue_init(m->q);
11 }
12
13 void lock(lock_t *m) {
14     while (TestAndSet(&m->guard, 1) == 1)
15         ; // 通过自旋获取 guard 锁
16     if (m->flag == 0) {
17         m->flag = 1; // 获取了锁
18         m->guard = 0;
19     } else {
20         queue_add(m->q, gettid());
21         m->guard = 0;
22         park();
23     }
24 }
25
26 void unlock(lock_t *m) {
27     while (TestAndSet(&m->guard, 1) == 1)
28         ; // 通过自旋获取 guard 锁
29     if (queue_empty(m->q))
30         m->flag = 0; // 放开锁；没人要它
31     else
32         unpark(queue_remove(m->q)); // 继续持有锁
33                                     //  （留给下一个线程!）
34     m->guard = 0;
35 }
```

图 28.9：**使用队列、测试并设置、让出和唤醒的锁**

后让出 CPU。留给读者一个问题：如果把释放 guard 锁的操作放在 `park()` 之后而不是之前，会发生什么？提示：会出问题。

你可能还会发现，另一个线程被唤醒时，标志并没有被设回 0。这是为什么？其实这不是错误，而是必须这么做！线程被唤醒时，就像是从 `park()` 返回一样；但这时它在代码中并不持有 guard，因此根本没法尝试把标志设为 1。所以，我们直接把锁从释放它的线程交给下一个获取它的线程；在这中间，`flag` 不会被设为 0。

最后，你可能注意到了，这个方案在调用 `park()` 之前似乎存在一个竞态条件（race condition，也叫竞争条件）。如果时机恰好不对，一个线程正准备 park，以为自己应该一直睡到锁不再被持有为止。这时如果切换到另一个线程（比如持有锁的线程），就可能出问

补充：避免自旋的又一个理由：优先级反转

避免使用自旋锁的一个很好的理由是性能：正如正文所说，如果一个线程在持有锁时被中断，其他使用自旋锁的线程就会花大量 CPU 时间，仅仅是在等锁变得可用。不过，事实证明，在某些系统上避免使用自旋锁还有另一个有意思的理由：正确性。需要提防的问题叫作**优先级反转**（priority inversion）。可惜，这是一场星际灾难，在地球 [M15] 和火星 [R97] 上都发生过！

假设系统中有两个线程。线程 2（T2）的调度优先级高，线程 1（T1）的优先级较低。在这个例子中，假设只要两个线程都可以运行，CPU 调度程序就总是先运行 T2；只有当 T2 无法运行时（比如 T2 因等待 I/O 而阻塞时），T1 才会运行。

问题来了。假设 T2 因为某种原因被阻塞了。于是 T1 运行，获取了一个自旋锁，进入临界区。这时 T2 不再阻塞（也许是因为 I/O 完成了），CPU 调度程序立刻调度它（于是 T1 被换下）。T2 现在试图获取锁，但因为拿不到（T1 持有锁），它就一直自旋。由于这个锁是自旋锁，T2 会永远自旋下去，系统就挂住了。

可惜，仅仅不用自旋锁，并不能避免反转问题（唉）。设想有三个线程 T1、T2 和 T3，T3 的优先级最高，T1 的最低。现在设想 T1 获取了一个锁。接着 T3 开始运行，由于它的优先级比 T1 高，它立刻就运行了（抢占了 T1）。T3 试图获取 T1 持有的锁，但因为 T1 仍然持有它，T3 只能卡在那里等待。如果这时 T2 开始运行，它的优先级比 T1 高，因此它会运行。优先级高于 T2 的 T3 卡在那里等 T1，而既然 T2 在运行，T1 可能永远也运行不了。强大的 T3 无法运行，卑微的 T2 却霸占着 CPU，这难道不让人难过吗？高优先级早就不像过去那样吃香了。

解决优先级反转问题有多种办法。在由自旋锁引起问题的特定情况下，可以避免使用自旋锁（下面会进一步介绍）。更一般地，等待低优先级线程的高优先级线程，可以临时提升低优先级线程的优先级，让它得以运行，从而消除反转，这种技术叫作**优先级继承**（priority inheritance）。最后一种办法最简单：让所有线程的优先级都相同。

题，例如那个线程紧接着释放了锁。这样一来，第一个线程随后执行的 park 就（可能）永远睡下去了，这个问题有时被称为 **wakeup/waiting race**。

Solaris 增加了第三个系统调用 `setpark()` 来解决这个问题。线程调用这个例程，表示自己马上要 park 了。如果它随后碰巧被中断，而另一个线程在它真正调用 park 之前调用了 unpark，那么随后的 park 就会立即返回，不会睡眠。`lock()` 内部的代码修改很小：

```
1 queue_add(m->q, gettid());
2 setpark(); // 新代码
3 m->guard = 0;
```

另一种解决方案是把 guard 传进内核。这样，内核就可以采取预防措施，原子地释放锁，并让正在运行的线程出队。

 © 2008–25, ARPACI-DUSSEAU

28.15 不同的操作系统，不同的支持

到目前为止，我们看到了一种操作系统支持，它能帮助线程库构建更高效的锁。其他操作系统也提供类似的支持，只是细节各不相同。

例如，Linux 提供了 **futex**，它和 Solaris 的接口相似，但在内核里实现了更多功能。具体来说，每个 futex 都关联着一个特定的物理内存位置，以及一个内核队列（每个 futex 各有一个）。调用者可以根据需要，使用 futex 调用（下面会介绍）来睡眠和唤醒。

具体来说，有两个调用可用。调用 `futex_wait(address, expected)` 会让调用线程进入睡眠，前提是地址 `address` 处的值等于 `expected`。如果不相等，调用会立即返回。调用例程 `futex_wake(address)` 会唤醒一个在队列上等待的线程。Linux 互斥锁中这些调用的用法见图 28.10（第 16 页）。

这段代码出自 nptl 库（gnu libc 库的一部分）中的 `lowlevellock.h` [L09]，有几处值得一说。第一，它用一个整数同时记录锁是否被持有（整数的最高位）和锁上等待者的数

```
1  void mutex_lock (int *mutex) {
2    int v;
3    // 第 31 位原来是 0，我们拿到了互斥锁（快速路径）
4    if (atomic_bit_test_set (mutex, 31) == 0)
5      return;
6    atomic_increment (mutex);
7    while (1) {
8        if (atomic_bit_test_set (mutex, 31) == 0) {
9            atomic_decrement (mutex);
10           return;
11       }
12       // 必须等待。首先确保我们监视的 futex 值
13       // 是负的（已加锁）。
14       v = *mutex;
15       if (v >= 0)
16         continue;
17       futex_wait (mutex, v);
18   }
19 }
20
21 void mutex_unlock (int *mutex) {
22   // 当且仅当没有其他关心这个锁的线程时，
23   // 给计数器加上 0x80000000 的结果才是 0
24   if (atomic_add_zero (mutex, 0x80000000))
25     return;
26
27   // 还有其他线程在等待这个互斥锁，
28   // 唤醒其中一个。
29   futex_wake (mutex);
30 }
```

图 28.10：**基于 Linux futex 的锁**

 © 2008–25, ARPACI-DUSSEAU

量（其余所有位）。因此，如果锁的值是负数，锁就是被持有的（因为最高位被置位了，而这一位决定整数的正负）。

第二，这段代码展示了如何针对常见情况做优化，具体来说，就是锁没有争用的情况：只有一个线程获取和释放锁时，要做的工作非常少（加锁时做一次原子的位测试并设置，解锁时做一次原子加法）。

看看你能不能把这个“真实世界”中的锁剩下的部分琢磨透，弄明白它是怎么工作的。去做吧，成为 Linux 加锁的大师，或者至少成为一个书上叫你做什么就做什么的人[3]。

28.16 两阶段锁

最后再说一点：Linux 的方法带有一种老方法的影子。这种方法多年来时断时续地有人在用，至少可以追溯到 20 世纪 60 年代初的 Dahm 锁 [M82]，现在叫作**两阶段锁**（two-phase lock）。两阶段锁意识到自旋是有用的，尤其是在锁即将被释放的时候。所以在第一阶段，锁会自旋一段时间，希望能拿到锁。

然而，如果在第一个自旋阶段没有拿到锁，就进入第二阶段：调用者进入睡眠，等以后锁空闲了才被唤醒。上面的 Linux 锁就是这样一种锁，只不过它只自旋一次；更一般的做法是，在一个循环中自旋固定的一段时间，然后再借助 **futex** 支持进入睡眠。

两阶段锁又是**混合**（hybrid）方法的一个例子：把两个好主意结合起来，确实可能得到一个更好的主意。当然，是否真能如此，在很大程度上取决于许多因素，包括硬件环境、线程数和工作负载的其他细节。和往常一样，要做出一个适合所有可能用例的通用锁，是相当有挑战性的。

28.17 小结

上面的方法展示了如今真实的锁是怎样构建的：一些硬件支持（以更强大的指令的形式），加上一些操作系统支持（例如 Solaris 上的 `park()` 和 `unpark()` 原语，或者 Linux 上的 **futex**）。当然，细节各不相同，实现这类加锁的具体代码通常也都经过了高度调优。想了解更多细节，可以看看 Solaris 或 Linux 的代码库，它们读起来很有意思 [L09, S09]。另外，David 等人有一项出色的工作，比较了现代多处理器上的各种加锁策略 [D+13]。

[3] 比如买一本 OSTEP 的纸质书！虽然这本书在网上确实可以免费阅读，难道你不想在桌上放一本精装本吗？或者更好的是，买上十本，分给亲朋好友？也许再多买一本，用来砸敌人？（这本书很重，所以扔出去出奇地有效。）

参考文献

[D91] "Just Win, Baby: Al Davis and His Raiders" by Glenn Dickey. Harcourt, 1991. *The book about Al Davis and his famous quote. Or, we suppose, the book is more about Al Davis and the Raiders, and not so much the quote. To be clear: we are not recommending this book, we just needed a citation.*

[D+13] "Everything You Always Wanted to Know about Synchronization but Were Afraid to Ask" by Tudor David, Rachid Guerraoui, Vasileios Trigonakis. SOSP '13, Nemacolin Woodlands Resort, Pennsylvania, November 2013. *An excellent paper comparing many different ways to build locks using hardware primitives. Great to see how many ideas work on modern hardware.*

[D68] "Cooperating sequential processes" by Edsger W. Dijkstra. 1968. Available online here: http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD123.PDF. *One of the early seminal papers. Discusses how Dijkstra posed the original concurrency problem, and Dekker's solution.*

[H93] "MIPS R4000 Microprocessor User's Manual" by Joe Heinrich. Prentice-Hall, June 1993. Available: http://cag.csail.mit.edu/raw/documents/R4400_Uman_book_Ed2.pdf. *The old MIPS user's manual. Download it while it still exists.*

[H91] "Wait-free Synchronization" by Maurice Herlihy. ACM TOPLAS, Volume 13: 1, January 1991. *A landmark paper introducing a different approach to building concurrent data structures. Because of the complexity involved, some of these ideas have been slow to gain acceptance in deployment.*

[L81] "Observations on the Development of an Operating System" by Hugh Lauer. SOSP '81, Pacific Grove, California, December 1981. *A must-read retrospective about the development of the Pilot OS, an early PC operating system. Fun and full of insights.*

[L09] "glibc 2.9 (include Linux pthreads implementation)" by Many authors.. Available here: http://ftp.gnu.org/gnu/glibc. *In particular, take a look at the nptl subdirectory where you will find most of the pthread support in Linux today.*

[M82] "The Architecture of the Burroughs B5000: 20 Years Later and Still Ahead of the Times?" by A. Mayer. 1982. Available: www.ajwm.net/amayer/papers/B5000.html. *"It (RDLK) is an indivisible operation which reads from and writes into a memory location." RDLK is thus test-and-set! Dave Dahm created spin locks ("Buzz Locks") and a two-phase lock called "Dahm Locks."*

[M15] "OSSpinLock Is Unsafe" by J. McCall. mjtsai.com/blog/2015/12/16/osspinlock-is-unsafe. *Calling OSSpinLock on a Mac is unsafe when using threads of different priorities – you might spin forever! So be careful, Mac fanatics, even your mighty system can be less than perfect…*

[MS91] "Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors" by John M. Mellor-Crummey and M. L. Scott. ACM TOCS, Volume 9, Issue 1, February 1991. *An excellent and thorough survey on different locking algorithms. However, no operating systems support is used, just fancy hardware instructions.*

[P81] "Myths About the Mutual Exclusion Problem" by G.L. Peterson. Information Processing Letters, 12(3), pages 115–116, 1981. *Peterson's algorithm introduced here.*

[R97] "What Really Happened on Mars?" by Glenn E. Reeves. Available on our site at: https://pages.cs.wisc.edu/%7Eremzi/OSTEP/Citations/mars.html. *A description of priority inversion on Mars Pathfinder. Concurrent code correctness matters, especially in space!*

[S05] "Guide to porting from Solaris to Linux on x86" by Ajay Sood, April 29, 2005. Available: http://www.ibm.com/developerworks/linux/library/l-solar/.

[S09] "OpenSolaris Thread Library" by Sun.. Code: src.opensolaris.org/source/xref/onnv/onnv-gate/usr/src/lib/libc/port/threads/synch.c. *Pretty interesting, although who knows what will happen now that Oracle owns Sun. Thanks to Mike Swift for the pointer.*

[W09] "Load-Link, Store-Conditional" by Many authors.. en.wikipedia.org/wiki/Load-Link/Store-Conditional. *Can you believe we referenced Wikipedia? But, we found the information there and it felt wrong not to. Further, it was useful, listing the instructions for the different architectures:* `ldl_l`/`stl_c` *and* `ldq_l`/`stq_c` *(Alpha),* `lwarx`/`stwcx` *(PowerPC),* `ll`/`sc` *(MIPS), and* `ldrex`/`strex` *(ARM). Actually Wikipedia is pretty amazing, so don't be so harsh, OK?*

[WG00] "The SPARC Architecture Manual: Version 9" by D. Weaver, T. Germond. SPARC International, 2000. http://www.sparc.org/standards/SPARCV9.pdf. See the website: developers.sun.com/solaris/articles/atomic_sparc/ for more on atomics.

作业（模拟）

程序 x86.py 可以让你看到，线程的各种交错执行方式是怎样导致或避免竞态条件的。程序如何工作详见 README，然后回答下面的问题。

问题

1. 看看 flag.s。这段代码用一个内存标志“实现”了加锁。你能看懂这段汇编吗？
2. 用默认设置运行时，flag.s 能正常工作吗？用 -M 和 -R 选项跟踪变量和寄存器（并打开 -c 查看它们的值）。你能预测 flag 最终会是什么值吗？
3. 用 -a 选项修改寄存器 %bx 的值（例如，如果只运行两个线程，就用 -a bx=2,bx=2）。这段代码做了什么？它会怎样改变你对上一题的回答？
4. 把每个线程的 bx 都设成一个较大的值，然后用 -i 选项产生不同的中断频率。哪些值会导致糟糕的结果？哪些会带来好的结果？
5. 现在来看程序 test-and-set.s。首先，试着理解这段代码，它用 xchg 指令构建了一个简单的加锁原语。获取锁是怎么写的？释放锁呢？
6. 现在运行这段代码，再次改变中断间隔（-i）的值，并确保循环若干次。代码总能按预期工作吗？它有时会让 CPU 的使用效率变低吗？你怎样量化这一点？
7. 用 -P 选项生成针对加锁代码的特定测试。例如，运行这样一个调度：在第一个线程中获取锁，然后在第二个线程中尝试获取它。结果对吗？还应该测试什么？
8. 现在来看 peterson.s 中的代码，它实现了 Peterson 算法（正文的补充框中提到过）。研究这段代码，看看你能不能弄懂它。
9. 现在用不同的 -i 值运行这段代码。你看到了哪些不同的行为？由于代码依赖线程 ID，一定要正确设置它们（例如用 -a bx=0,bx=1）。
10. 你能控制调度（用 -P 选项）来“证明”这段代码是正确的吗？你应该证明哪些情况成立？想想互斥和避免死锁（deadlock）。
11. 现在研究 ticket.s 中票锁的代码。它和本章中的代码一致吗？然后用以下选项运行：-a bx=1000,bx=1000（让每个线程把临界区循环执行 1000 次）。观察会发生什么：线程花在自旋等待锁上的时间多吗？
12. 增加线程数时，代码的行为会怎样？
13. 现在看看 yield.s，其中一条 yield 指令让一个线程交出对 CPU 的控制（现实中这应该是一个操作系统原语，但为简单起见，我们假设由一条指令来完成这个任务）。找出一种场景，在这种场景下 test-and-set.s 会把周期浪费在自旋上，而 yield.s 不会。节省了多少条指令？这种节省会出现在哪些场景中？
14. 最后，看看 test-and-test-and-set.s。这个锁做了什么？与 test-and-set.s 相比，它带来了哪些节省？

 © 2008–25, ARPACI-DUSSEAU